Julia port of the FSA-v2 model: closed-loop SMC^2 filter + GPU controller

Coding session writeup

2026-05-07

1 The Julia FSA-v2 codebase

1.1 Repository layout

The model lives at `version_2_Julia/models/fsa_high_res/`. Eight files, mirroring the Python source under `version_2/models/fsa_high_res/`.

File	Purpose
`_dynamics.jl`	SDE drift + state-dependent diffusion (Stage G1 reparametrisation).
`_phi_burst.jl`	Daily-Φ $\rightarrow$ sub-daily morning-loaded burst envelope.
`simulation.jl`	`INIT_STATE`, `DEFAULT_PARAMS`, the four observation samplers, circadian $C(t)$.
`_plant.jl`	`StepwisePlant` — `advance!` (daily Φ + burst envelope) and `advance_subdaily!` (per-bin Φ direct). `finalise` writes the psim artefact.
`cpu_control.jl`	CPU `build_control` — RBF + cost functional, debug reference only.
`estimation.jl`	30 priors + 5 frozen, the locally guided (Pitt–Shephard) `propagate_fn`, observation log-weights, alignment grid.
`gpu_pf.jl`	FSA-specific GPU bootstrap-PF kernel + `FSAGPUTargetBatched`.
`gpu_control.jl`	FSA-specific GPU cost kernel `fsa_cost_kernel!` + `FSAControlGPUTarget` + `make_log_density_fn` closure.
`FSAHighRes.jl`	Module aggregator.

Framework additions (under `julia/SMC2FC/src/`, model-independent):

- `Filtering/GPUSegmentedPF.jl` — shared per-chain weight reductions, normalised-weight cumulative sum, systematic resample with Liu–West shrinkage, OT sigmoid-blend rescue.
- `Control/GPUControlSMC.jl` — generic parallel-chains tempered SMC^2, parallel-chains HMC kernel, ChEES-L picker, central-FD gradient batcher. Consumes a model-supplied closure `log_density_fn` : $\mathbb{R}^{M\times d} \rightarrow \mathbb{R}^M$.

1.2 Dynamics

State $\boldsymbol{X} = (B, F, A)$ evolves under the Itô SDE

$$\begin{aligned}
\mathrm{d}B &= \kappa_B \cdot a_B(A) \cdot \Phi \,\mathrm{d}t - \tfrac{B}{\tau_B}\,\mathrm{d}t + \sigma_B\sqrt{B(1-B)}\,\mathrm{d}W_B \\
\mathrm{d}F &= \kappa_F \Phi \,\mathrm{d}t - a_F(A)\,\tfrac{F}{\tau_F}\,\mathrm{d}t + \sigma_F\sqrt{F}\,\mathrm{d}W_F \\
\mathrm{d}A &= \mu(B,F)\,A\,\mathrm{d}t - \eta A^3\,\mathrm{d}t + \sigma_A\sqrt{A}\,\mathrm{d}W_A,
\end{aligned}$$

with the operating-point reparametrisation $a_B(A) = (1+\varepsilon_A A)/(1+\varepsilon_A A_{\text{typ}})$, $a_F(A)$ the analogue for λ_A, and $\mu(B,F) = \mu_0 + \mu_B B - \mu_F F - \mu_{FF}(F - F_{\text{typ}})^2$ with $A_{\text{typ}} = 0.10$, $F_{\text{typ}} = 0.20$. Truth values use the effective parameters: $\kappa_B^{\text{eff}} = 0.01248$, $\tau_F^{\text{eff}} = 6.364$, $\mu_F^{\text{eff}} = 0.26$, $\mu_0^{\text{eff}} = 0.036$.

1.3 Plant

`StepwisePlant` carries the truth state (B, F, A) and a global bin counter. `advance!` expands daily Φ values via the morning-loaded Gamma envelope (peak around 10 am, zero overnight, integral preserved at $24{\cdot}\Phi_d$/day) and runs single-step Euler-Maruyama with the sqrt-Itô diffusion above. `advance_subdaily!` skips the burst expansion and applies a caller-supplied per-bin Φ directly. Both leave the same `history` of trajectory + obs that `finalise` dumps to disk in psim format.

1.4 Estimation

The 30 estimated parameters are declared as a `Vector{Tuple{Symbol,PriorType}}` ordered identically to Python's `PARAM_PRIOR_CONFIG` so the 30-panel posterior trace plot lines up column-by-column. `propagate_fn` runs the locally guided (Pitt–Shephard) proposal: prior predictive $\mathcal{N}(\mu_{\text{pred}}, P_{\text{pred}})$ from the G1 drift and state-dependent variance, then sequential scalar Kalman fusion across the three Gaussian channels (HR, stress, log(steps+1)) gated by `*_present` masks, then Cholesky-3 sample. `obs_log_weight_fn` adds the Bernoulli sleep log-likelihood. Verbatim port of Python's `estimation.py:propagate_fn`.

1.5 GPU particle filter

Model file contains only the FSA-specific propagate kernel — same locally guided proposal as the CPU version, written as a KernelAbstractions `@kernel` that runs in fp32 with one thread per (chain, particle). Resampling, Liu–West shrinkage and OT rescue are framework calls (`Filtering/GPUSegmentedPF.jl`). Public entry point: `gpu_log_density_batched(target, U)`, returning the per-chain marginal log-likelihood.

1.6 GPU controller

The control kernel `fsa_cost_kernel!` runs one thread per (chain m, common-random-numbers trial t). Per outer bin k: decode $\Phi(t_k) = \Phi_{\max}\sigma\big(c_\Phi + \sum_a \theta_{m,a}\,\varphi_a(t_k)\big)$ from the (raw, non-normalised) Gaussian RBF basis; pre-step accumulate the cost integrand $A_{\text{acc}} \mathrel{+}= A\,\Delta t$, $B_{\text{acc}} \mathrel{+}= \max(F - F_{\max}, 0)^2\,\Delta t$; then run the substepped EM SDE step under the trial-shared CRN noise. Cost is Eq 37 of the spec verbatim:

$$J(\boldsymbol{\theta}_{\text{ctrl}}) \;=\; -\int_0^T A(t)\,\mathrm{d}t \;+\; \lambda_F \int_0^T \max\big(F(t) - F_{\max},\, 0\big)^2\,\mathrm{d}t, \qquad \text{(Eq. 37)}$$

with $\lambda_F = 1$, $F_{\max} = 0.40$. The model exposes a `make_log_density_fn(target)` closure that hands the per-chain MC-averaged $-J$ to the framework's `run_tempered_smc_gpu`.

1.7 Configuration and wall time

The reference run — $T = 14\,\mathrm{d}$, $h = 15\,\mathrm{min}$, replan every stride, $n_{\mathrm{SMC}}^{\mathrm{filter}} = 32$ inner-PF chains $\times K = 400$ particles, $n_{\mathrm{SMC}}^{\mathrm{ctrl}} = 1024 \times n_{\mathrm{inner}} = 128$ trials, HMC $\varepsilon = 0.2$ / $L_{\mathrm{ChEES}} \in \{16, 32, 64, 128, 256\}$ / `num_mcmc`=10, target $\beta_{\max} = 8$ nats — completes in ∼780 s on a single RTX 5090. Python's reference run at the same configuration takes ∼27 minutes.

2 Bug hunt: what's been ruled out, what remains

The Julia controller settles on a *flat low-*Φ schedule ($\Phi \in [0.10, 0.30]$ across all 27 strides) with mean autonomic amplitude $\bar{A}_{\mathrm{MPC}} = 0.085$ vs $\bar{A}_{\mathrm{baseline}} = 0.082$ (a +3.7% gain). Python on the same model reports a *recovery → overload* schedule (Φ goes $1.0 \to 0.2 \to 1.2$) with $\bar{A}_{\mathrm{MPC}} = 0.122$ (a +50% gain). The qualitative shape of the controller's policy is wrong, not just the magnitude.

This section is the diagnostic timeline. Each hypothesis was tested by one targeted experiment.

2.1 Hypothesis 1 (kept): RBF basis was row-normalised

The original Julia `build_rbf` row-normalised the design matrix into a partition of unity. Python's `smc2fc/control/rbf_schedules.py:design_matrix` returns the *raw* Gaussian basis (no normalisation). With row-normalisation the decoded $\Phi(t)$ collapses to a weighted average of θ_a values, smoothing out any per-anchor variation regardless of the gradient signal.

Test: removed normalisation in both `cpu_control.jl:build_rbf` and the GPU target's RBF construction. **Outcome:** Φ moved from a flat ~ 0.45 to a flat ~ 0.20, matching the spec's text description of a "rest-leaning $\Phi \in [0.25, 0.31]$". Real fix, kept in code. *It did not produce the recovery → overload pattern, but it was a real bug.*

2.2 Hypothesis 2 (kept): three small kernel mismatches vs Python

Found while reading the Julia kernel against Python's `control.py:cost_fn` line-by-line:

1. A NaN guard at the end of every bin reset the state to $(0.05, 0.30, 0.10)$ if any of B, F, A became non-finite. Removed — the boundary reflection already handles the legitimate cases, and the guard was masking real numerical issues.
2. The cost diffusion used $\sqrt{A + \varepsilon_A}$ ($\varepsilon_A = 10^{-4}$). Python's controller path uses $\sqrt{\max(A, 0)}$ (the ε_A form lives only in the plant's `noise_scale_fn`). Aligned.
3. The cost integrand A_{acc} was being accumulated AFTER the EM step. Python uses the PRE-step state. Switched.

Outcome: small numerical changes, no qualitative effect on the controller's optimum. Kept anyway — they bring the kernel into bit-for-bit correspondence with Python's algorithm.

2.3 Hypothesis 3 (discarded): bursty / smooth Φ asymmetry between MPC and baseline plants

The bench was calling `advance!` (which expands daily Φ through the burst envelope) for the baseline plant and `advance_subdaily!` (which takes per-bin Φ directly) for the MPC plant. Under constant $\Phi = 1$, a Monte-Carlo plant test gave $\bar{A}_{\mathrm{bursty}} = 0.082$ vs $\bar{A}_{\mathrm{smooth}} = 0.100$ — a 22 % gap that suggested the MPC trajectory was being measured on a different fast-dynamics regime from the baseline.

Test: aggregated the controller's per-bin Φ to daily values and routed both plants through `advance!` (so both run on the bursty envelope). **Outcome:** the bench produced essentially

identical numbers ($\bar{A}_{\mathrm{MPC}} = 0.085$, Φ still flat at ~ 0.20). The baseline-plant asymmetry was real and worth fixing, but it was *not* the source of the controller's wrong optimum. *Hypothesis discarded.*

2.4 Hypothesis 4 (discarded): bug in the framework controller

Suspicion: the parallel-chains tempered SMC^2 + ChEES-HMC machinery in `julia/SMC2FC/src/Control/GPUControlSMC.jl` might have a gradient-following bug or a tempering-bisection bug that prevents it from finding off-prior-mean optima.

Test: ran the framework controller against three synthetic analytic costs with *known* optima. Pure CPU log-density closures ($\mathbb{R}^{M\times d} \to \mathbb{R}^M$), no model and no GPU kernel. Driver: `version_2_Julia/tools/test_framework_controller.jl`.

- **Quadratic bowl** $J(\boldsymbol{\theta}) = \frac{1}{2}\|(\boldsymbol{\theta} - \boldsymbol{\theta}^\star)/\sigma_J\|^2$ with $\boldsymbol{\theta}^\star \neq \mathbf{0}$. Framework returned posterior mean $0.755 \cdot \boldsymbol{\theta}^\star$, matching the analytic Bayesian-shrinkage prediction at the auto-calibrated $\beta_{\max} = 1.37$ within rounding. *Pass.*
- **Anti-symmetric reward** $J(\boldsymbol{\theta}) = -\boldsymbol{w}^\top\boldsymbol{\theta} + \gamma\|\boldsymbol{\theta}\|^2$ with $\boldsymbol{w}$ anti-symmetric across the eight RBF anchors (negative early, positive late) — the same spatial structure as the FSA recovery $\to$ overload schedule. Framework returned a posterior mean with the correct anti-symmetric signs and ratios, shrunk by $\sim 25\%$ — same Bayesian-shrinkage signature. *Pass: the framework finds spatially structured optima when they exist.*
- **Bimodal cost** with one "flat" mode at $\boldsymbol{\theta}_a = -\mathbf{1}$ and one "recovery $\to$ overload" mode at $\boldsymbol{\theta}_b = (-1.5, -1.5, -1.5, 0, 0, +1.5, +1.5, +1.5)$, with $\boldsymbol{\theta}_b$ the global mode. Tested three initialisations: from prior, from $\boldsymbol{\theta}_a$, from $\boldsymbol{\theta}_b$. All three converged to the same prior-weighted compromise — not a defect, but the consequence of $\beta_{\max}$ auto-calibrating to 0.367 under the wide prior (the prior dominates the tempered posterior at small β).

Outcome: the framework's controller logic is correct; in particular it can find anti-symmetric optima when they are present (Test 2). *Hypothesis discarded.*

2.5 Hypothesis 5 (discarded): HMC stuck in the wrong basin on FSA

Even if the cost surface contained both a flat-low and a recovery $\to$ overload basin and the latter were deeper, an HMC chain initialised in the larger flat-low basin might never escape.

Test: ran the FSA bench with the controller's HMC initialised at a constructed recovery $\to$ overload $\boldsymbol{\theta}_{\mathrm{seed}}$ (per-anchor target $\Phi = (1.0, 0.2, 0.2, 0.4, 0.7, 0.9, 1.1, 1.2)$, decoded through the inverse RBF formula, $\sigma_{\mathrm{jitter}} = 0.3$ around the seed). Same controller config, same wall ($\sim 740\,\mathrm{s}$).

Outcome: every replan still settled at $\bar{\Phi} \in [0.27, 0.30]$, identical final state $\hat{x} = (0.064, 0.071, 0.096)$ to the cold-start runs. HMC initialised at the recovery $\to$ overload point *moved away* from it and converged to flat-low. The recovery $\to$ overload region is not a local minimum at all in the Julia FSA cost surface — the gradient there points toward flat-low. *Hypothesis discarded.*

2.6 Hypothesis 6 (kernel + decoder OK on monotone reward)

A trivial-cost end-to-end test that isolates the FSA SDE + GPU kernel + RBF decoder + framework HMC, removing all the Stuart–Landau and F-coupling nonlinearities of the original cost.

Test cost. Replace Eq 37 by

$$J(\mathbf{\Phi}) \;=\; -\int_0^T B(t)\,\mathrm{d}t, \qquad \lambda_F = 0 \quad \text{(no fatigue penalty)}.$$

Under the FSA SDE, $\mathrm{d}B/\mathrm{d}t = \kappa_B \cdot a_B(A) \cdot \Phi - B/\tau_B$ is monotone in Φ. The optimum is to pin $\Phi = \Phi_{\max} = 3$ for every bin, driving B toward its highest equilibrium $\kappa_B\,\Phi_{\max}\,\tau_B$.

Implementation. The cost kernel is parametrised with state-integrand weights (w_B, w_F, w_A) (default $(0, 0, 1)$, i.e. Eq 37). Setting $(1, 0, 0)$ and $\lambda_F = 0$ gives the trivial cost. Driver: `version_2_Julia/tools/test_max_B_only.jl`.

Result. The framework returns posterior-mean $\bar{\boldsymbol{\theta}} = (+1.57, +2.12, +1.92, +1.77, +1.71, +1.52, +1.04, +0.42)$ which decodes to a per-bin Φ schedule with:

- day 0 to day 8: $\Phi(t) \in [2.68, 2.92]$ — essentially pinned to $\Phi_{\max}$.
- day 10 to day 14: Φ tapers down to 1.79 — Bayesian shrinkage near the boundary anchors where the prior pull is more visible.
- summary: $\Phi_{\text{mean}} = 2.73$ vs $\Phi_{\max} = 3.0$.

Six tempering levels, auto-calibrated $\beta_{\max} = 12.5$ (large because $\int B$ has small variance under the prior cloud, so cost-std is small), 55 s wall.

Conclusion. When the optimum is monotone in Φ, the controller finds it. End-to-end this rules out:

- Bug in the FSA SDE integration ($\mathrm{d}B/\mathrm{d}t$ is correct).
- Bug in the RBF decoder ($\theta_a > 0$ correctly produces high Φ at anchor a).
- Bug in the GPU-kernel cost-accumulation pathway ($\int B\,\mathrm{d}t$ is correctly assembled).
- Bug in the framework parallel-chains HMC.

The remaining root cause is therefore confined to the A-cost surface specifically — the Stuart–Landau cubic $-\eta A^3$, the F-coupling through $\mu(B, F)$, or some numerical interaction between them that displaces the global optimum from recovery → overload to flat-low in Julia versus Python.

2.7 Hypothesis 7 (root cause confirmed): fp32 cancellation in the G1 drift

Triggered by the observation that the v1 reference controller (`version_1_Julia/models/fsa_high_res/gpu_control.jl`, working end-to-end on the same SDE structure) uses the *pre-G1* drift form, while my v2 uses the G1-reparametrised form. The two are mathematically equivalent at the truth parameters — v1 mu and v2 mu both evaluate to -0.031 at $(B, F, A) = (0.05, 0.30, 0.10)$. The G1 reparametrisation cannot be unwound on the v2 side because v2 needs it for FIM-rank identifiability of the static parameters in the *filter*. So the controller has to use the v2 form.

The v2 form contains arithmetic that is more cancellation-prone in fp32 than the v1 form:

- $(F - F_{\text{typ}})^2$ subtracts magnitudes of order $F_{\text{typ}} = 0.20$ then squares the small remainder, losing fp32 mantissa bits.
- $(1 + \varepsilon_A A)/(1 + \varepsilon_A A_{\text{typ}})$ divides near-equal numbers when $A \approx A_{\text{typ}}$. Both numerator and denominator are ≈ 1.04; the ratio is determined by their small difference. Same for the λ_A term in $a_F(A)$.

- Larger individual terms in μ that have to cancel: $\mu_F^{\text{eff}} F = 0.078$ at $F = 0.30$ vs $\mu_F^{\text{v1}} F = 0.030$.

Each substep evaluation of the drift accumulates rounding from these sources. Over the controller's cost rollout ($n_{\text{steps}} \times n_{\text{substeps}} = 1344 \times 4 = 5376$ evaluations per trial, $n_{\text{inner}} = 128$ trials per chain), the biased rounding shifts integrated A by enough to displace the global cost minimum from the recovery→overload schedule to flat-low.

Test. The drift block inside the substep loop of `gpu_control.jl:fsa_cost_kernel!` was promoted to fp64 (state and accumulators stay fp32; only the per-substep arithmetic that produces `drift_B`, `drift_F`, `drift_A` runs in fp64). A single planning call at the canonical init state $(B, F, A) = (0.05, 0.30, 0.10)$ over the full $T = 14$ d horizon was run, with a light controller config ($n_{\text{SMC}} = 64$, $n_{\text{inner}} = 16$, `num_mcmc` $= 3$, $L_{\text{leap}} = 8$) to keep wall time tractable on consumer fp32:fp64 silicon (RTX 5090's fp64 throughput is $\sim 1/64$ of fp32). Driver: `version_2_Julia/tools/test_one_plan_fp64.jl`.

Result. Posterior-mean $\bar{\boldsymbol{\theta}}$ decoded to:

day 0	$\Phi = 0.23$	day 8	$\Phi = 0.31$
day 1	$\Phi = 0.13$	day 9	$\Phi = 0.36$
day 2	$\Phi = 0.10$	day 10	$\Phi = 0.42$
day 3	$\Phi = 0.10$	day 11	$\Phi = 0.52$
day 4	$\Phi = 0.12$	day 12	$\Phi = 0.67$
day 5	$\Phi = 0.15$	day 13	$\Phi = 0.84$
day 6	$\Phi = 0.20$	day 14	$\Phi = 0.98$
day 7	$\Phi = 0.25$		

This is the *recovery → overload* pattern. Φ drops to its floor in days 1–3 (clearing the high initial fatigue), holds low through day 5–6, then ramps monotonically up to nearly $\Phi_{\max}/3$ by day 14. Compare to the fp32 production run, which gave a flat $\Phi \approx 0.20$ across all 14 days.

The auto-calibrated $\beta_{\max}$ also moved dramatically: 117 in fp64 vs ∼12 in fp32, indicating the cost has $\sim 10\times$ larger variance across the prior cloud once the cancellation is removed — the cost surface is genuinely sharp, the fp32 noise was washing it out.

Hypothesis confirmed. The flat-low controller behaviour was caused by accumulated fp32 cancellation in the v2 G1 drift.

2.8 Outstanding: production-grade fix

The fp64-promotion test confirms the diagnosis but is not a production fix — it costs $\sim 30\times$ more wall-time on the 5090 (consumer Blackwell's fp64 throughput is 1/64 of fp32). The full bench at production config ($n_{\text{SMC}} = 1024, n_{\text{inner}} = 128$) would take $\sim$ 15 hours under fp64 substep arithmetic.

The proper fix is an algebraic reformulation of the v2 G1 drift that avoids the cancellation hot-spots while staying mathematically identical:

- Rewrite $\mu = \mu_0 + \mu_B B - \mu_F F - \mu_{FF}(F - F_{\text{typ}})^2$ as a Horner-style polynomial in F that fuses the constant terms: $\mu = (\mu_0 - \mu_{FF} F_{\text{typ}}^2) + \mu_B B + (2\mu_{FF} F_{\text{typ}} - \mu_F) F - \mu_{FF} F^2$. This is identically the v1 polynomial in F once the constant and linear coefficients are pre-combined — an algebraic rearrangement that the JIT may or may not perform on its own.
- Pre-compute $a_{\text{typ}}^{-1} \equiv 1/(1 + \varepsilon_A A_{\text{typ}})$ once at target construction, then evaluate $a_B(A) = (1 + \varepsilon_A A) \cdot a_{\text{typ}}^{-1}$ — one multiply, no division, no near-equal subtraction.

- Same trick for $a_F(A)$.

After the reformulation, the production fp32 kernel should reach the same $\beta_{\max}$ and the same posterior-mean schedule as the fp64 diagnostic, at the original $\sim 800\,\mathrm{s}$ wall time.

Update 2026-05-07 (post-application). The Horner reformulation and the precomputed a_{typ}^{-1} factors were applied to `gpu_control.jl:fsa_cost_kernel!` (commit in working tree). Three verifications followed:

- Bit-for-bit at $\theta = 0$ and the hand-crafted θ_{RO} — Julia GPU-fp32 and Python fp64 still agree to a max relative difference of 3×10^{-6}.
- Light-config open-loop (n_{SMC}=64, n_{inner}=16): the controller still finds the recovery→overload optimum, Φ ramps $0.10 \to 0.95$ over 14 d, ∼15 s wall.
- **Heavy-config closed-loop (n_{SMC}=1024, n_{inner}=128, 27 replans): the bench still settles to flat-low.** Per-replan plan means $\bar{\Phi}_{\mathrm{plan}} \in [0.26, 0.39]$ (recovery→overload-shaped *plans*), but cumulative applied Φ stays in $[0.10, 0.35]$ and final $\bar{A}_{\mathrm{MPC}} = 0.085$ vs baseline 0.082 — the same flat-low numbers as before §2.8. 701 s wall time, matching the reference.

Conclusion: §2.8 alone is necessary but not sufficient. The cancellation hot-spots are gone (which is why the light-config open-loop test continues to recover the right optimum), but the heavy closed-loop is still broken. See §2.10 for the standing hypothesis and the staged plan that supersedes the single §2.8 fix.

2.9 Summary of what was ruled out, and what fixed it

Hypothesis	Test	Outcome
1. RBF row-norm	Removed, re-ran bench	Real bug. Kept.
2. Kernel mismatches (NaN guard, ε_A, POST-step)	Aligned to Python, re-ran	Cosmetic. Kept.
3. Bursty/smooth Φ asymmetry	Routed MPC plant through `advance!`	No effect. Discarded.
4. Framework controller bugs	3 synthetic costs with known optima	Framework correct.
5. HMC stuck in wrong basin	Seeded HMC at recovery→overload	HMC drifts away. Discarded.
6. SDE / decoder / kernel infrastructure	Max-$\int B$ trivial reward	$\Phi \to \Phi_{\max}$ correctly.
7. fp32 cancellation in G1 drift	fp64-promoted substep arithmetic	Recovery→overload restored at light config.
8. §2.8 algebraic rearrangement (Horner $\mu + a_{\text{typ}}^{-1}$)	Applied; bit-for-bit + light open-loop + heavy closed-loop	Light open-loop OK; heavy closed-loop still flat-low. §2.8 was necessary but not sufficient.
9. Bench was using shrinking remaining-bench horizon as planning horizon	Replaced with fixed $H_{\text{plan}} = T_{\text{bench}}$ at every replan (mirrors Python)	Real bug. Applied Φ now has Python's qualitative U-shape (1.0 day 0, 0.2 mid, 0.78 by day 13). *But* amplitude is too low: $\bar{A}_{\text{MPC}}$ moved only from 0.085 to 0.087 vs Python's 0.122. Necessary, not sufficient.
10. Why does Julia undershoot Python's $\bar{A}$ even with right shape?	Apply Python's exact daily-Φ schedule to Julia's plant (`tools/test_apply_python_schedule.jl`).	**Plant-design difference, by design.** Julia uses one 24-bin daily envelope; Python uses two 12-bin per-stride envelopes. Same daily-Φ → different sub-day F trajectories → Julia's $\bar{A} = 0.087$ vs Python's 0.122 under *identical* schedules. Accepted: Julia's plant is the intended design.

The cancellation hot-spots in the v2 G1 drift's $(F - F_{\text{typ}})^2$ and $(1 + \varepsilon_A A)/(1 + \varepsilon_A A_{\text{typ}})$ are gone after §2.8. The light-config open-loop test confirms the cost-surface shape is right at

$n_{\text{inner}} = 16$. But at production $n_{\text{inner}} = 128$ the heavy closed-loop bench still settles to flat-low, so a residual fp32 bias must survive elsewhere in the kernel. §2.10 lays out the staged-promotion plan for finding the minimum sufficient fp64 promotion that fixes the heavy closed-loop.

2.10 Hypothesis 8 (in progress): residual fp32 bias surviving §2.8 — staged fp64 promotion

Standing hypothesis after §2.8 was applied and didn't fix the closed-loop: the Horner reformulation removed the worst (cancellation- prone) fp32 ops, but smaller systematic biases survive in the still-fp32 parts of the kernel. The mechanism that's consistent with all the observed numbers:

- At **light config** ($n_{\text{inner}} = 16$) the per-cost MC noise is $\sim \sigma/\sqrt{16}$. This noise dominates whatever systematic fp32 bias is left, so the cost surface seen by HMC is approximately the same as Python's at low signal-to-noise — the global recovery→overload basin still wins.
- At **heavy config** ($n_{\text{inner}} = 128$) the MC noise shrinks by $\sqrt{8} \approx 2.8\times$. The systematic fp32 bias that was buried in noise becomes the dominant distortion, and because it's plan-dependent it warps the cost surface enough to displace the global optimum.

This is consistent with: bit-for-bit at probe θ values (where the cost is point-evaluated and noise/bias both irrelevant) matches Python; light-config open-loop matches Python; heavy-config closed-loop diverges.

Three candidate locations for the residual fp32 bias. Inside `gpu_control.jl:fsa_cost_kernel!` after §2.8, the remaining fp32 paths are:

1. **Cost accumulators** (`A_acc`, `barrier_acc`). Sum of 1344 fp32 increments to a total reaching ~ 1.4. By the tail of the sum the relative add ratio $|A\,\Delta t|/|A_{\text{acc}}| \approx 10^{-3}$ — below the fp32 mantissa floor for the smallest dropped bits. The bias is *plan-dependent* (plans driving A higher accumulate more bits of loss), so it distorts the cost *surface*, not just the level. **Cheapest to fix** — two scalar promotions per thread, no per-substep cost.
2. **State accumulation**, $B \mathrel{+}= \Delta t \cdot \dot{B}$ etc., repeated 5376 substeps. Same compounding-rounding pattern, on the state itself. If state drifts off, downstream A diverges.
3. μ_{bif} **Horner sum** — four signed terms of similar magnitude added in fp32 each substep. Less likely to be plan-dependent at the magnitudes involved (all four terms $\mathcal{O}(0.01\text{–}0.1)$) but worth checking if (1)+(2) don't suffice.

Staged methodology. Promote one location at a time, measure each step against three verifications:

Step 1. Restore the §2.8 production baseline (fp32 + Horner) and re-confirm the three known outcomes: bit-for-bit at probe θ values $\leq 3 \times 10^{-6}$; light open-loop recovery→overload; heavy closed-loop flat-low.

Step 2. Promote **cost accumulators** to fp64 only. Verifications: bit-for-bit (should improve slightly), light open-loop (must remain recovery→overload), heavy closed-loop (the diagnostic — does Φ shift?).

Step 3. If Step 1 didn't shift the heavy closed-loop, additionally promote the **state** (B, F, A) to fp64. Same verification suite.

Step 4. If Step 2 didn't shift it either, additionally promote the **drift computation** (μ_{bif}, a_{factor}, the drift outputs) to fp64. This is the upper bound; the failed full-fp64 conversion already showed open-loop matches Python at this point, so heavy closed-loop must too — otherwise the framing of "residual fp32 bias displaces the optimum" was wrong and the bug is somewhere orthogonal (orchestration, plant ↔ kernel mismatch, etc.).

Step 5. Pick the cheapest set that produces recovery→overload in the heavy closed-loop and lock it in with a comment block citing the empirical comparison.

Stop conditions: stop early if Step 1 already fixes the heavy bench (ship it); stop and reassess if any step regresses the open-loop test (means the change broke something orthogonal); stop after Step 3 regardless and shift the investigation back to orchestration.

Verification target. The Julia heavy closed-loop trace plot `E5_full_mpc_T14d_traces.png` should qualitatively match the Python reference at `version_2/outputs/fsa_high_res/g4_runs/T14d_replanK2_h60min_no_infoaware/E5_full_mpc_T14d_traces.png`: applied Φ visibly U-shaped (high day-0, low days 1–5, ramping back up to ~ 1.0 by day 13), F decaying then turning back up at the overload phase, $\bar{A}_{\text{MPC}} \gtrsim 0.115$ (Python target 0.122), F-violation $\leq 1\%$.

The corresponding implementation plan is `claude_plans/Localising_residual_fp32_bias_in_the_v2_GPU_cost_kernel_2026-05-07_1956.md`.

2.11 Hypothesis 9 (real bug, partial fix): receding-horizon MPC was using the wrong horizon

A real MPC formulation bug, fixed — but not the full root cause. The bench's per-replan plan was being constructed over the *shrinking remaining-bench horizon*, not over a *fixed planning horizon*. These are two distinct concepts that the original code conflated:

- T_{bench} — the closed-loop *bench duration* (= 14 d here). How long the experiment runs.
- H_{plan} — the controller's *planning horizon*. How far ahead the controller looks at each replan. Should be HELD FIXED throughout the bench at a value chosen for the dynamics (here $H_{\text{plan}} = \tau_B = 14\,\text{d}$, one chronic-B time constant — enough lookahead for the recovery→overload trade-off to be visible).

The original Julia bench tied $H_{\text{plan}} = T_{\text{bench}} - t_{\text{elapsed}}$. By stride 20 of a 14-day bench (= day 10), H_{plan} had shrunk to 4 days. The cost-surface optimum at the current state (low F, moderate B) over a 4-day window collapses to "recover" — there's no time left for the controller to commit to a final overload phase that would actually pay back in $\int A\,\mathrm{d}t$. Plan day-0 stays at the recovery start ($\Phi \approx 0.20$); the bench commits this and replans the next stride from a state still in the recovery basin; same plan shape; same $\Phi \approx 0.20$ committed; ... for the rest of the bench. Cumulative applied Φ stuck in [0.10, 0.30].

Python's `bench_smc_full_mpc_fsa.py` line 389 hard-codes `plan_horizon_days=T_total_days` at every replan: a FIXED 14-day lookahead, regardless of how much bench remains. At late strides the plan extends *past* the bench's end; only the first stride is committed before the next replan, so the past-the-end portion is just discarded. With a long-enough lookahead, the cost optimum from the late-bench state (low F, moderate B) is the U-shape: overload immediately (because F is low and there's room to push B further), brief recovery, then a second overload. Plan day-0 = high; applied Φ rises to ~ 1.2 at the bench's end.

Verification by isolated open-loop test. A 14-day open-loop plan from the late-bench state $(B, F, A) = (0.075, 0.092, 0.090)$ with the heavy controller config returns posterior-mean $\bar{\theta} = (+0.40, -0.08, -0.66, -0.87, -0.95, -0.62, -0.42, -0.17)$ which decodes to Φ at days $\{0, 2, 4, 6, 8, 10, 12, 14\} = \{1.21, 0.82, 0.39, 0.24, 0.23, 0.33, 0.50, 0.73\}$ — the *U-shape* that matches Python's reference plot. Plan day-0 = 1.21. From the same state, a 4-day open-loop plan gives plan day-0 ≈ 0.22 (recovery). The cost optimum's plan day-0 is plan-horizon-dependent in exactly the way the failure mode predicts.

The partial fix. `version_2_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl` now replans with `ctrl_n_steps = round(args["T-days"] / DT_DAYS)` at every replan boundary, regardless of `mpc_plant.t_bin`. The plan covers a fixed lookahead; only the first stride is committed. Combined with Hypothesis 8's end-of-stride replan strategy (initial baseline $\Phi = 1.0$ for the first K strides, then replan every K), the closed-loop bench's applied Φ now has the qualitative U-shape that matches Python's reference — $\Phi = 1.0$ on day 0, drop to ~ 0.2 during recovery (days 2–5), ramp up to ~ 0.78 by day 13.

What's still wrong. The applied Φ shape is right but its amplitude is too low, and the A trajectory does not climb to Python's level.

- Python's reference: applied Φ peaks at 1.2 by day 13, A(MPC) peaks at 0.17, $\bar{A}_{\text{MPC}} = 0.122$ vs baseline 0.081 (+51% gain).
- Julia after this fix: applied Φ peaks at 0.78 by day 13, A(MPC) stays in $[0.08, 0.10]$ and actually *drops* during recovery, $\bar{A}_{\text{MPC}} = 0.087$ vs baseline 0.085 (a +2.3% gain).

So the controller is shaping the schedule correctly (recovery followed by overload) but is committing too little overload — not enough late-phase Φ to drive B high enough to lift $\mu(B, F)$ into the regime where $A^* = \sqrt{\mu/\eta}$ exceeds the initial $A = 0.10$. There is at least one more bug between the controller and the plant; §2.11 picks up the investigation.

What was ruled out by getting here. §2.1 – §2.7 had narrowed the bug to "somewhere in the cost surface near the A-cost integrand under the v2 G1 drift", and §2.8 / §2.10 tried algebraic and staged-fp64 fixes. None of those touched the closed-loop optimum. §2.9 (this section) is a real bug — the receding-horizon was using the wrong horizon, and the fix materially shifts the closed-loop schedule into the right qualitative shape — but it does not fully reproduce Python's gain. So the kernel, the framework, the precision setup, and §2.8 are all correct (open-loop tests confirm); §2.9's MPC formulation is also now correct; but at least one more bug is still affecting the closed-loop trajectory.

2.12 Hypothesis 10 (root cause): bench was feeding the plant a re-bursted daily-average of the controller's smooth schedule

The actual bug, found by side-by-side reading against the ground-truth driver `tools/test_max_A_with_F_barrier.jl`: the bench was taking the controller's per-bin Φ slice for each stride, *averaging it to a single daily value*, and then *re-expanding it through the morning-loaded burst envelope* before passing to the plant.

```
# wrong (bench, before fix):
daily_phi_for_stride = mean(phi_stride)     # destroy controller's shape
advance!(mpc_plant, advance_bins, [daily_phi_for_stride])  # re-burst
```

That replaces the controller's actual schedule with a different signal: a daily-averaged Φ refracted through a Gamma-shaped morning burst. The plant then runs an SDE under that

re-bursted signal, not the smooth schedule the controller's cost kernel was optimising against. Pure plant ↔ controller mismatch.

The ground-truth driver (§2.10 above) calls `advance_subdaily!` with the controller's per-bin Φ directly:

```
# ground truth (correct):
advance_subdaily!(mpc_plant, Float32.(slice))  # controller's exact phi(t)
```

Same call now in the bench:

```
# bench, after fix:
advance_subdaily!(mpc_plant, Float32.(phi_stride))
advance_subdaily!(base_plant, fill(Float32(1.0), advance_bins))
```

After this fix the bench's mean A_{MPC} is essentially the same as the ground-truth open-loop driver's (0.087 vs 0.091, within MC noise — the ground truth has cleaner numbers because it commits one plan rather than re-optimising every K strides). The qualitative Φ trace now has the U-shape Python's reference shows (baseline at days 0–1, recovery to $\Phi \approx 0.2$ days 2–5, ramp with peaks reaching $\Phi \approx 1.4$ by day 10).

Why this is the actual root cause. The diagnostic chain through §2.1–§2.9 was repeatedly hitting a "flat-low applied Φ" symptom. Every time we fixed something upstream of the plant (RBF norm, kernel mismatches, fp32 cancellation, algebraic Horner form, planning horizon, replan strategy) the applied schedule got closer to recovery→overload but A never grew. With the plant call corrected, the controller's smooth $\Phi(t)$ now actually drives the plant. The A/B test on burst-envelope modes (`--plantsingle_daily` vs `per_stride`) gave the same mean $A = 0.087$ in both modes *because both modes are wrong* relative to what the controller expects — the controller wants a smooth signal, not any flavour of bursty refraction. The fix is to drop the bursty refraction entirely and pass the per-bin Φ as-is.

Status. This was the residual bug after §2.9. The full bug list:

1. §2.1 RBF row-normalisation (kept fix — real bug).
2. §2.2 NaN guard / ε_A / POST-step accumulation (kept fixes — cosmetic).
3. §2.9 Receding-horizon used shrinking remaining-bench horizon (kept fix — real bug).
4. §2.10 (this section) Bench was re-bursting the controller's smooth schedule before passing to the plant (kept fix — real bug, the residual bottleneck).

§2.3, §2.4, §2.5, §2.6 were ruled-out hypotheses (controller machinery is fine). §2.7, §2.8 / §2.11 chased an fp32 cancellation hypothesis that the staged-promotion test (Step 1 fp64 accumulators, Step 2-alt Kahan + FMA) ruled out — the algebraic-stable kernel is kept anyway as a free correctness win, but it was not load-bearing for the optimum.

Open-loop mode in the bench. After fixing the plant call, the closed-loop bench's applied Φ still oscillates because each replan re-finds a slightly different optimum, and the cost surface has multiple plausible basins HMC can land in. To verify the bench is now structurally equivalent to the ground-truth driver, the bench has an `--open-loop true` flag that disables replanning: ONE plan up front from INIT_STATE+TRUTH_PARAMS, applied for the whole bench. With this flag the bench produces the smooth recovery→ramp Φ that the ground-truth driver does ($\Phi : 0.20 \to 0.12 \to 0.78$ over 14 d), and $\bar{A}_{\mathrm{MPC}}$ within MC noise of the ground truth's value (0.085 vs 0.091).

The closed-loop schedule oscillation under the corrected plant call is a residual MPC-policy concern (each replan can find a different local optimum), not a kernel/plant/orchestration bug. It is out of scope for the bug-hunt; the canonical reference for this codebase is now the open-loop result.

2.13 Hypothesis 11 (open-loop matches; closed-loop oscillates): replanning is the residual issue

With §2.9 (fixed planning horizon) and §2.10 (`advance_subdaily!`) both applied, two regimes can be tested side by side:

Open-loop bench (`--open-loop true`). A new CLI flag in `tools/bench_smc_full_mpc_fsa_gpu.jl` disables replanning: the controller computes ONE plan up front from INIT_STATE+TRUTH_PARA the plant applies it through 27 strides via `advance_subdaily!`, the filter still runs but no replan is triggered. This mirrors the structure of the ground-truth driver `tools/test_max_A_with_F_barrier.jl` exactly. Result at $h = 15$ min:

- Applied Φ: smooth recovery→ramp $0.20 \to 0.12 \to 0.78$ over 14 d, no oscillation. Same qualitative shape as the ground- truth driver.
- $\bar{A}_{\text{MPC}} = 0.085$, $\bar{A}_{\text{baseline}} = 0.082$, A trajectory dips to ∼0.07 during recovery and recovers to ∼0.10 by day 14. Within MC noise of the ground-truth driver's $\bar{A}_{\text{MPC}} = 0.091$.

So the kernel + plant + framework SMC^2 + HMC are all working when exercised by the same single-plan structure as the ground truth. The canonical reference for this codebase is now this open-loop result.

Closed-loop bench at heavy config. The default mode — replan at end of every K strides, fixed planning horizon, posterior-mean dynamics params from the filter — still produces an oscillating applied Φ across the bench, with peaks swinging between $\Phi = 0.5$ and $\Phi = 1.4$. $\bar{A}_{\text{MPC}}$ is also around 0.087 (within MC noise of open-loop), so the magnitude gap to Python is the same; the visual difference is the noise in the Φ trace. Two contributions identified:

1. **Filter posterior is noisy at $n_{\text{smc}} = 32$.** The diagnostic prints in `bench_smc_full_mpc_fsa_gpu.jl` show posterior-mean μ_F swinging from 0.244 to 0.261 (5–7% stride-to-stride) and κ_B swinging from 0.0119 to 0.0130. Each replan plans from a slightly different params vector, which can land HMC in a different local minimum of the cost surface. Python's reference uses $n_{\text{smc\_particles}} = 1024, n_{\text{pf\_particles}} = 800$ for the filter; Julia's bench uses $n_{\text{smc}} = 32, K_{\text{per\_chain}} = 400$. About 64× fewer particles.
2. **Even with TRUTH_PARAMS forced** (filter posterior bypassed for the controller's params), replan-by-replan plans still land in different basins because each replan starts from a different `last_xhat` (filter's smoothed state estimate), and the cost surface near the heavy-config MAP appears multimodal (open-loop bisection from a low-F state can find plan day-0 $\in [0.7, 1.4]$ depending on RNG seed). At $n_{\text{smc}} = 32$ HMC's chains are also jittery, which is on top of the filter-state jitter.

Why this is in the closed-loop bench but not the ground-truth driver. The ground truth never replans, so the controller commits one plan in one cost-surface basin. The closed-loop bench replans 13 times in a 27-stride run; each replan has to find a fresh MAP. The local-minimum-spread combined with filter-state jitter is what manifests as the oscillating applied schedule.

2.14 Performance: Julia under-utilises the RTX 5090 at large n_{smc}

A practical observation while testing fixes for §2.11. With $n_{\text{smc}} = 32$ the bench finishes in ~ 220 s; with $n_{\text{smc}} = 1024$ (matching Python's reference filter configuration) the bench had not finished a single stride after ~ 140 s of warmup, with GPU utilisation around 51 % and only ~ 7 GB / 32 GB of memory used. Linear extrapolation gave a $\sim$ 60–80 min total. Python at the same config takes ~ 27 min, so Julia is roughly 2–3× *slower* than Python at large n_{smc} — the wrong direction for a pure-GPU port.

The plausible bottlenecks (both visible by reading `run_outer_smc_gpu` in `bench_smc_full_mpc_fsa_gpu.jl` and `parallel_hmc_one_move!` in `models/fsa_high_res/gpu_pf.jl`):

- Each tempering level (10–11 per stride) calls `gpu_log_density_batched` once for the cost evaluation and then `num_mcmc` $\times L$ more times inside the HMC moves for the FD gradient batcher (`gpu_grads_parallel_chains` expands M chains to $M(1 + 2d) = 61M$ rows, runs the inner-PF on all of them, returns to host). Each call has a host-side per-chain CPU loop that re-builds the params matrix (`params_cpu`), an unconditional `KernelAbstractions.synchronize` at every segment of the inner-PF, and a small but per-chain `Array(view(...))` copy back from GPU into a host array for the segmented log-likelihood accumulation.
- Filter resampling and OT rescue paths run a CPU-side per-chain branch (line 216, 252 of `Filtering/GPUSegmentedPF.jl`).
- HMC's accept/reject and posterior cloud update are on the host in a per-particle loop (`parallel_hmc_one_move!`:527–533).

None of these are intrinsically GPU-bound limitations; they are host-side overhead per-chain that scales with n_{smc} even though the work could be batched. So the practical fix is either (a) batch these more aggressively, removing the host-side per-chain loops, or (b) accept the $n_{\text{smc}} = 32$ scale and look for filter stabilisation that doesn't require more particles (e.g. stronger warm-start bridge across windows, or carrying the controller forward without re-pulling from the filter posterior at every replan). This is in scope for the next iteration but out of scope for the bug-hunt that was supposed to be a pure "why does Python work and Julia not?" diagnosis.

2.15 Updated summary table

Hypothesis	Test	Outcome
1. RBF row-norm	Removed, re-ran bench	Real bug. Kept.
2. Kernel mismatches (NaN guard, ε_A, POST-step)	Aligned to Python, re-ran	Cosmetic. Kept.
3. Bursty/smooth Φ asymmetry	Routed MPC plant through `advance!`	No effect. Discarded.
4. Framework controller bugs	3 synthetic costs with known optima	Framework correct.
5. HMC stuck in wrong basin	Seeded HMC at recovery→overload	HMC drifts away. Discarded.
6. SDE / decoder / kernel infrastructure	Max-$\int B$ trivial reward	$\Phi \to \Phi_{\max}$ correctly.
7. fp32 cancellation in G1 drift	fp64-promoted substep arithmetic	Recovery→overload restored at light config.
8. Algebraic rearrangement (Horner $\mu + a_{\text{typ}}^{-1}$)	Applied; bit-for-bit + light open-loop + heavy closed-loop	Necessary, not sufficient. Kept.
9. Receding-horizon used shrinking remaining-bench horizon	Replaced with fixed $H_{\text{plan}} = T_{\text{bench}}$ at every replan	Real bug. Kept.
10. Bench was averaging controller's per-bin Φ to a daily and re-bursting through plant envelope	Replaced with `advance_subdaily!` (controller's smooth $\Phi(t)$ goes straight to plant)	**Real bug; the residual bottleneck through §2.9.** Open-loop now matches ground truth.
11. Closed-loop replan oscillation	Diagnostic prints + open-loop comparison	Open-loop matches ground truth. Closed-loop oscillates due to filter-posterior + cost-surface basin sensitivity. Magnitude gap unchanged.

3 Outstanding

- Decide whether to ship the bench as "open-loop matches ground truth, closed-loop is residually noisy" or to fix the closed-loop oscillation. The cheapest closed-loop stabiliser would be

carrying the controller's posterior forward through a Gaussian bridge across replans (§7 of the framework reference) so HMC warm-starts in the same basin instead of cold-starting from the prior every K strides.

- Fix the GPU-utilisation regression (§2.12). Specifically: remove the per-chain host loops in `run_outer_smc_gpu`, `gpu_log_density_batched`, `run_segmented_smc_step!`, and `parallel_hmc_one_move!` by either pre-allocating a single device buffer for the params / accumulators, or moving the per-chain branches inside a single `@kernel` so the GPU stays saturated as n_{smc} grows.
- Once the controller is producing a stable schedule, switch the FD gradient to ForwardDiff dual-number autodiff. Per the bistable B3 module's empirical note, ForwardDiff with $d = 8$ out-performs Enzyme reverse-mode by $\sim 2.7\times$ on a similar PF; the FSA controller has the same dimensionality. The seeded-HMC diagnostic of §2.5 showed gradient quality wasn't the bug, but exact gradients buy a $17\times$ throughput improvement per HMC move (one autodiff pass vs $1 + 2d = 17$ FD evaluations).

4 What is in the bench today

`version_2_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl` implements:

- **§2.8 algebraic-stable kernel** in `models/fsa_high_res/gpu_control.jl`: Horner-form μ, precomputed a_{typ}^{-1}, Kahan-compensated cost accumulators, FMA throughout the sub-step drift. All fp32 except the per-trial cost output. Bit-for-bit-matches Python at probe θ values to 3×10^{-6} relative.
- **§2.9 fixed planning horizon**: every replan plans `T-days` ahead, regardless of how much bench is left. Mirrors Python's `plan_horizon_days=T_total_days`.
- **§2.10 `advance_subdaily!` plant call**: the controller's per-bin Φ goes straight to the plant via `advance_subdaily!` without any daily-averaging or burst-envelope re-application. Plant integrates the SDE bin-by-bin under the exact signal the cost kernel optimised against.
- **§2.9 end-of-stride replan strategy**: apply baseline $\Phi = 1.0$ for the first K strides, then replan every K strides at end-of-stride. Mirrors Python's `if (s+1) % K == 0`.
- `--open-loop true` **flag**: disables replanning, applies a single up-front plan from INIT_STATE+TRUTH_P for the whole bench. Reproduces the ground-truth driver structurally.
- **Auto-plot hook** (with `Base.invokelatest` world-age workaround) that regenerates `E5_full_mpc_T*_traces.png` and the 30-panel param-trace plot on every bench finish.

The closed-loop bench at default config (`--replan-K2--step-minutes60`) reproduces the qualitative U-shape of Python's applied Φ but with an oscillating overlay; `--open-loop true` reproduces a smooth recovery→ramp that matches the ground-truth driver within MC noise.

5 Running experiments

This section is for a new agent / user picking up the codebase. Every experiment in this writeup was produced by one Julia driver:

`version_2_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl`

5.1 Prerequisites

- **Working directory.** All commands assume you are in `version_2_Julia/` (so that `Project.toml` and `tools/` are at the right relative paths).
- **Julia 1.11+** with the project's environment activated:

```
cd version_2_Julia
julia --project=. -e "using Pkg; Pkg.instantiate()"
```

- **NVIDIA GPU** (codebase tested on RTX 5090). The KernelAbstractions kernels target CUDABackend.
- **No conda env / Python** required. The bench is pure Julia. Comparisons against Python use saved `.npz` outputs in `version_2/outputs/...`.

5.2 The minimal invocation

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl
```

Defaults: $T = 14\,\mathrm{d}$, $h = 15\,\mathrm{min}$, replan every stride ($K = 1$), $n_{\mathrm{smc}} = 32$, $K_{\mathrm{per\_chain}} = 400$, num_mcmc = 5, hmc step size 0.025, hmc leapfrog 8, seed 42. Total wall $\approx 700\,\mathrm{s}$ on the RTX 5090. Output goes to `outputs/fsa_high_res/g4_runs/T14d_replanK1_h15min_no_infoaware/`.

The auto-plot hook (§4 above) writes `E5_full_mpc_T14d_traces.png` (4-panel B/F/A/Φ trace) and `E5_full_mpc_T14d_param_traces.png` (30-panel filter posterior trace) into the same output dir before the bench exits.

5.3 Reproducing Python's reference run (matched config)

Python's reference plot `version_2/outputs/fsa_high_res/g4_runs/T14d_replanK2_h60min_no_infoaware/` was produced at $h = 60\,\mathrm{min}$, $K = 2$. To match Python's outer config:

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
  --replan-K 2 --step-minutes 60
```

Total wall $\approx 200\,\mathrm{s}$.

5.4 Reproducing the ground-truth driver structurally

If you want the smooth recovery→ramp Φ trace that matches `tools/test_max_A_with_F_barrier.jl` (§2.11 above), use open-loop mode:

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
  --open-loop true --step-minutes 15
```

Total wall $\approx 260\,\mathrm{s}$. Disables replanning entirely; one plan up front from INIT_STATE+TRUTH_PARAMS is committed for the whole bench.

5.5 All CLI flags

Every flag has the form `--name <value>`. Values are parsed according to the type of the default:

Flag	Default	Meaning
`--T-days`	`14` (Int)	Bench duration. Plant rolls forward this many days under the controller's schedule. The planning horizon at every replan is also `T-days` (§2.9).
`--step-minutes`	`15` (Int)	Outer time step in minutes. Must divide 1440. Determines BINS_PER_DAY = 1440/step-minutes. Read at module load via `ENV["FSA_STEP_MINUTES"]`, so this must be parsed *before* the model imports (the driver does this). Common values: `15` (high-res, default), `60` (matches Python).
`--replan-K`	`1` (Int)	Replan every K strides at end-of-stride. The first K strides apply baseline $\Phi = 1.0$ (mirrors Python's `daily_phi_plan = np.full(T_total_days, baseline)` initialiser). Then replan happens at `(s % K) == 0`. Common values: `1` (replan every stride), `2` (Python's default). Ignored when `--open-loop true`.
`--N-smc`	`32` (Int)	Number of outer-SMC^2 particles for the FILTER. Each particle spawns its own inner-PF. Wall-time scales roughly linearly with this value at the current Julia implementation (see §2.12 — there's a known performance regression with per-chain host loops). Python's reference uses `1024`.
`--K-per-chain`	`400` (Int)	Inner-PF particles per filter chain. Total filter particles = `N-smc` × `K-per-chain`. Python's reference uses `800`.
`--num-mcmc`	`5` (Int)	Number of HMC moves per tempering level inside the FILTER's outer SMC^2. Increase if HMC acceptance is too high (chain not mixing). The CONTROLLER's num_mcmc is hard-coded to `ctrl_num_mcmc = 10` in the bench (line 322).
`--hmc-step-size`	`0.025` (Float)	FILTER's HMC leapfrog step size. The CONTROLLER uses `ctrl_hmc_step = 0.2` (hard-coded, line 324) — different from the filter because the controller's θ_{ctrl} has much wider scale.
`--hmc-leapfrog`	`8` (Int)	FILTER's leapfrog trajectory length. CONTROLLER uses `ctrl_hmc_leap = 16` (hard-coded).
`--max-lambda-inc`	`0.10` (Float)	Cap on $\delta\lambda$ per tempering level for the filter's bisection. Smaller = finer schedule, more levels, safer.
`--target-ess-frac`	`0.5` (Float)	Target effective-sample-size fraction for the filter's tempering bisection. Lower = larger $\delta\lambda$ per level, fewer levels but riskier.
`--max-temp-levels`	`30` (Int)	Hard cap on filter tempering levels. The bench currently logs all levels per stride.
`--seed`	`42` (Int)	Master RNG seed. Plants use `seed_offset = --seed`; controller noise grids use `seed + 100 + s` per stride.
`--output-dir`	`""` (String)	Output directory for `data.jld2`, `manifest.json`, `experiment_run.md`, and the auto-generated PNG plots. If empty (default), the bench builds an auto-named path: `outputs/fsa_high_res/g4_runs/T<days>d_replanK<K>_h<step>min_no_infoaware/`. Pass an explicit directory to keep separate runs side-by-side.
`--open-loop`	`"false"` (String)	Set to `true` to disable replanning entirely. The bench computes ONE plan up front from INIT_STATE

5.6 Hard-coded controller knobs (not currently CLI-exposed)

These are inside `tools/bench_smc_full_mpc_fsa_gpu.jl` around line 318–328 and have to be edited in source if you want to change them:

- `ctrl_n_smc = 1024` — controller's outer-SMC^2 particles. Already at Python's reference value.
- `ctrl_n_inner = 128` — MC trials per cost evaluation (= number of CRN noise realisations averaged for each θ sample's cost). Python's reference value.
- `ctrl_n_anchors = 8` — number of Gaussian RBF anchors for the schedule decoder. Determines $\theta_{\text{ctrl}} \in \mathbb{R}^8$.
- `ctrl_target_ess_frac = 0.5`
- `ctrl_num_mcmc = 10`
- `ctrl_max_lambda_inc = 0.20`
- `ctrl_hmc_step = 0.2`
- `ctrl_hmc_leap = 16`
- `ctrl_max_levels = 30`
- `ctrl_target_nats = 8.0` — target nat budget for the controller's auto-calibrated $\beta_{\max}$.
- `ctrl_sigma_prior = 1.5` — prior std on θ_{ctrl}.

5.7 What lands on disk after a run

In the output directory:

- `data.jld2` — raw data the plot scripts consume: `trajectory_mpc`, `trajectory_baseline` (both $(n_{\text{bins}}, 3)$ for B, F, A), `Phi_per_bin_mpc` + `Phi_per_bin_baseline`, `daily_phi_per_stride`, `posterior_particles` ($(n_{\text{strides}}, n_{\text{smc}}, 30)$), `param_names`, `truth_params`, plus run metadata.
- `E5_full_mpc_T<days>d_traces.png` — 4-panel diagnostic: B, F, A trajectories (MPC vs constant-Φ baseline) + applied daily Φ schedule.
- `E5_full_mpc_T<days>d_param_traces.png` — 30-panel filter posterior trace plot, one panel per static dynamics parameter.
- `manifest.json` — machine-readable run config (seed, config knobs, wall time, etc.).
- `experiment_run.md` — human-readable summary.

If the auto-plot fails (e.g. KernelAbstractions world-age issue), the `data.jld2` is still written. To re-generate plots from saved `data.jld2`:

```
julia --project=. tools/plot_state_traces.jl \
  <out_dir>/data.jld2 <out_dir>/E5_full_mpc_T14d_traces.png

julia --project=. tools/plot_param_traces.jl \
  <out_dir>/data.jld2 <out_dir>/E5_full_mpc_T14d_param_traces.png \
  14 15 48
```

(arguments to `plot_param_traces.jl`: `T-days`, `step-minutes`, `stride-bins`.)

5.8 Suggested experiments

1. **Sanity check the codebase.**

   ```
   julia --project=. tools/test_cost_at_theta0.py    # Python side
   cd version_2_Julia
   julia --project=. tools/test_cost_at_theta0.jl    # Julia side
   ```

   Confirms the cost kernel matches Python fp64 to 3×10^{-6} at probe θ values.

2. **Fastest closed-loop bench.**

   ```
   julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
     --replan-K 2 --step-minutes 60
   ```

   $\sim$ 200 s. Default config that matches Python's outer-loop config (replan-K=2, h=60min). Currently the closed-loop applied Φ oscillates per §2.11.

3. **Smooth recovery→ramp (open-loop).**

   ```
   julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
     --open-loop true --step-minutes 15
   ```

   $\sim$ 260 s. Reproduces the ground-truth driver's smooth schedule.

4. **High-res, replan every stride.**

   ```
   julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl   # all defaults
   ```

   $\sim$ 700 s. $h = 15$ min, $K = 1$.

5. **Stronger filter (warning: slow).**

   ```
   julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
     --replan-K 2 --step-minutes 60 --N-smc 256
   ```

   Filter at $n_{\text{smc}} = 256$ (8× default). Wall scales approximately linearly — expect $\sim$ 30 min until §2.12 is fixed.

6. **Plant-only sanity check** (apply Python's exact daily Φ to Julia's plant; quantifies the plant-design gap):

   ```
   julia --project=. tools/test_apply_python_schedule.jl
   ```

   $\sim$ 10 s. Confirms Julia's plant under Python's schedule produces $\bar{A}_{\text{MPC}} \approx 0.087$ vs Python's 0.122.

5.9 Saturating the RTX 5090: which flags to push

Default config gets the RTX 5090 to about 50 % utilisation with $\sim$ 7 GB of 32 GB used — there is a lot of headroom. §2.12 explains why blindly bumping `--N-smc` doesn't proportionally speed things up (per-chain host loops). But for a novice agent who wants to push more work onto the GPU and watch utilisation rise, here are the flags that actually scale GPU work, in order of impact:

1. `--N-smc` **(filter outer-SMC^2 particles).** Most direct lever. Each filter particle spawns its own inner-PF run, so doubling `N-smc` doubles the total work the GPU has to do per stride. The inner-PF kernel launches are with $M \times K$ threads; bigger M = bigger launches = better occupancy. Combined with the FD gradient batcher ($M(1 + 2d) = 61M$ chains per HMC step), `N-smc=128` already produces $128 \times 61 \times K = 488$k-particle launches with $K = 400$, comfortable for the 5090.
   Try: `--N-smc 32` (default), then `64`, `128`, `256`, `512`. Each step doubles wall. Watch `nvidia-smi` util go up.

2. `--K-per-chain` **(inner-PF particles per chain).** Total filter work = `N-smc` × `K-per-chain`. Bumping K is sometimes preferable to bumping N because it widens the per-launch thread count without amplifying host-side per-chain loops as much.
   Try: `--K-per-chain 400` (default), then `800` (Python's reference), `1600`.

3. `--step-minutes` **(outer time step).** Smaller step = more bins per stride = more SDE work per kernel launch. $h = 15$ min produces 96 bins/day; $h = 5$ min produces 288 bins/day (3× work). The substep loop inside the cost kernel and the inner-PF propagate kernel both scale linearly with bins.
   Try: `--step-minutes 60` (smallest work), `15` (default), `5` (3× wall vs default). Caveat: only values that evenly divide 1440 are accepted.

4. `--T-days` **(bench duration).** More strides = more total benches' worth of work. Doesn't widen parallelism per launch, but the controller's planning horizon scales with it (`ctrl_n_steps` = `T-days` × `BINS_PER_DAY`) so each *controller* call is bigger.
   Try: `--T-days 14` (default), `42` (one Banister chronic time constant — single canonical horizon), `84` (two chronic constants).

5. **Hard-coded controller knobs** (need editing the bench source, lines 318–328): `ctrl_n_inner`, `ctrl_n_smc`, `ctrl_hmc_leap`, `ctrl_num_mcmc`. These already default to Python's heavyweight reference values ($n_{\text{inner}} = 128$, $n_{\text{smc}} = 1024$, $L = 16$, mcmc= 10), so the controller side already saturates fairly well. Increasing `ctrl_n_inner` beyond 128 widens each cost evaluation (more CRN trials averaged per θ).

6. **Less impactful for saturation** but increases total compute (and wall): `--num-mcmc` (more HMC moves per filter tempering level — sequential, doesn't widen parallelism), `--hmc-leapfrog` (longer leapfrog trajectory — same gradient batch, just more iterations), `--max-temp-levels` (caps filter levels, usually doesn't bind anyway).

Recommended novice sweep. In one terminal start a watcher:

```
watch -n 0.5 nvidia-smi --query-gpu=utilization.gpu,memory.used \
                        --format=csv
```

In another, sweep the filter dimensions while keeping everything else default ($h = 60$ min, $K = 2$):

```
for N in 32 64 128 256; do
  for K in 400 800 1600; do
    julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
      --replan-K 2 --step-minutes 60 \
      --N-smc $N --K-per-chain $K \
      --output-dir outputs/sweep/N${N}_K${K}
  done
done
```

Wall time scales roughly linearly with $N \times K$ at the current implementation; the sweep above is about 50 min total at $h = 60$ min. Read off util/memory from the watcher to see where saturation kicks in. The bench's auto-plot writes `E5_full_mpc_T14d_traces.png` per cell of the sweep, so you can also compare quality of the resulting controller schedule across $N \times K$.

What to look for.

- `utilization.gpu` should approach 90–100 % at the larger configs. If it plateaus at ~ 50 % regardless of $N \times K$, you have hit the per-chain host-loop ceiling diagnosed in §2.12 — bumping the flags more won't help; the fix is in the source (batch the per-chain loops).
- `memory.used` growing predictably with $N \times K \times d$ confirms the GPU is allocating per-particle buffers; if it does *not* grow with N, the bench is reusing buffers and the bottleneck is launch overhead, not capacity.
- Per-stride wall-time printed in the bench output (`[stride X/27] Y.Ys ...`) is the practical metric. At saturation, stride wall should grow sub-linearly with $N \times K$ (per-launch fixed costs amortise).

Don't bother trying these for saturation.

- `--replan-K`: only changes how often the controller fires, not how big each fire is.
- `--target-ess-frac`, `--max-lambda-inc`: tune the bisection schedule, change the number of tempering levels but not the work per level.
- `--seed`: changes RNG; same compute.
- `--open-loop true`: skips replanning so total compute drops. Use this for the smooth-schedule run, not for saturation testing.

5.10 Known gotchas

- **Working directory.** The driver `includes models/fsa_high_res/FSAHighRes.jl` and the plot scripts via paths relative to its own `@__DIR__`. You must run Julia from `version_2_Julia/` (or use an absolute path). If you run from the repo root, you'll get a `SystemError: opening file "../tools/..."` immediately.
- **Auto-plot world-age.** The bench's auto-plot block uses `Base.invokelatest` to dispatch to the included plot functions. If a future agent edits the plot files and adds new exports, those exports won't be visible to the bench's main() unless the `invokelatest` pattern is preserved. See bench lines 627–660.
- **One Julia process per bench.** KernelAbstractions caches compiled kernels per Julia process. Running multiple benches in parallel via `&` works but the second one pays its own JIT cost. For a sweep, prefer one Julia process invoking `main(args::Dict)` multiple times in a loop.
- **No comparison plot generator.** Comparing a Julia output against the Python reference requires running `tools/compare_to_python.jl` (which reads both `.npz` and `.jld2`). The bench itself doesn't do this.